

merox.dev

Infrastructura Kubernetes & VPS — Documentatie completa

Ghid complet al repo-ului infrastructure

Fiecare fisier. Fiecare linie importanta. Explicat in romana.

Ansible

Terraform

Talos

Flux GitOps

Cilium

Longhorn

SOPS

OpenClaw AI

Generat: 07 June 2026

github.com/meroxdotdev/infrastructure

Cuprins

1. Arhitectura Generala	4
Stack complet — de la hardware la aplicatii	4
Structura directorului radacina	4
2. Tooling si Configuratie Globala	5
.mise.toml — Versiunile uneltelor	5
.sops.yaml — Regulile de criptare	5
Taskfile.yaml — Entry points principale	5
3. VPS Layer — Ansible	8
vps/Makefile — Interfata principala	8
Playbook-uri Ansible principale	8
Vault Ansible — Secretele VPS-ului	8
Terraform VPS — Hetzner (DR fallback)	8
4. Talos Linux — OS-ul Kubernetes	10
talos/talconfig.yaml — Configuratia clusterului	10
talos/talenv.yaml — Versiunile	10
talos/patches/ — Modificarile fata de default	10
5. Terraform — Disaster Recovery VMs pe Proxmox	11
talos/terraform/main.tf — Ce se creeaza	11
talos/terraform/variables.tf — Parametrii configurabili	11
6. Bootstrap — Prima Pornire a Clusterului	12
bootstrap/helmfile.yaml — Ordinea de instalare	12
task bootstrap:talos — Bootstrap cluster Talos	12
7. Flux GitOps	13
Componente Flux si rolul lor	13
flux-instance/app/helm/values.yaml — Optimizarile Flux	13
Structura kustomization-urilor (ierarhia Flux)	13
8. Networking — Cilium (CNI + Gateway API)	14
cilium/app/helm/values.yaml — Configuratii cheie	14
Gateway API — doua Gateway-uri	14
9. Storage — Longhorn	15
longhorn/app/helmrelease.yaml — Setarile importante	15
longhorn/recurringjob/recurringjob.yaml — Backup automat	15
restore-pvs/pvs.yaml — PV-urile pentru DR restore	15
10. Certificate Management — cert-manager	16
ClusterIssuer letsencrypt-production	16
11. Aplicatii — Namespace default	17
Jellyfin — Media Server (HTPC)	17

Stack media: Sonarr + Radarr + Prowlarr + qBittorrent	17
Jellyseerr — Gestionar request-uri media	17
n8n — Automatizare (no-code)	17
Homepage — Dashboard	17
Authentik Outpost — SSO Proxy K8s	18
12. Observability — Monitorizare	19
kube-prometheus-stack	19
Grafana (10Gi PVC)	19
Loki + Promtail (20Gi PVC)	19
Netdata	19
13. Agenti AI — OpenClaw	20
agent/openclaw.json — Configuratia principala	20
Workspaces — Cei 8 agenti	20
agent/scripts/ — Infrastructura de rulare	20
agent/dashboard/scripts/ — Scripturile de update dashboard	20
14. Secrets Management — SOPS + age	21
Cum functioneaza criptarea	21
Tipuri de secrete in repo	21
Renovate Bot — Actualizare automata versiuni	21
15. Disaster Recovery — Procedura Completa	22
Scenariul 1 — VPS Oracle pica (Ansible + Hetzner)	22
Scenariul 2 — Cluster K8s pica (Talos + Flux + Longhorn)	22
Fisiere critice — TREBUIE backup-ate	22

1. Arhitectura Generala

Repo-ul infrastructure gestioneaza doua straturi principale ale infrastructurii merox.dev:

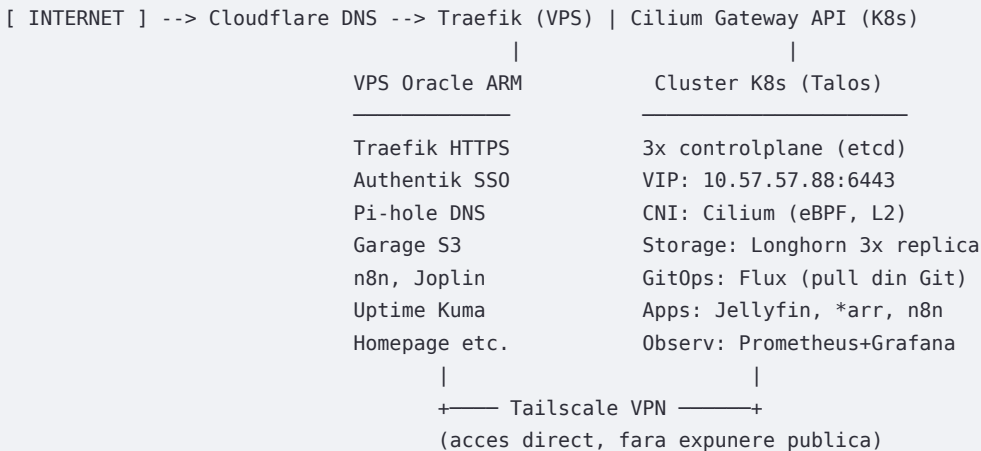
Strat 1 — VPS Oracle Cloud (Docker)

Un singur VPS ARM pe Oracle Cloud (4 vCPU / 24 GB RAM / Ubuntu 24.04). Gazduieste servicii Docker gestionate prin Ansible + Terraform. Punct de intrare public: Traefik, Authentik SSO, Pi-hole, Garage S3.

Strat 2 — Cluster Kubernetes on-premises (Talos + Flux)

3 noduri Talos Linux (controlplane x3) pe hardware fizic Proxmox. Flux GitOps sincronizeaza automat configuratiile din acest repo. Longhorn asigura storage distribuit cu backup automat in Garage S3.

► Stack complet — de la hardware la aplicatii



► Structura directorului radacina

Director / Fisier	Rol
vps/	Ansible + Terraform pentru VPS Oracle. Makefile = interfata principala.
talos/	Configuratii Talos Linux (talconfig.yaml) + patches + Terraform pentru DR VMs.
kubernetes/	Manifeste Flux GitOps: toate aplicatiile K8s, networking, storage, observability.
bootstrap/	helmfile.yaml — instalarea manuala a primelor componente (Cilium, Flux etc.).
agent/	Configuratie OpenClaw AI agents: workspaces, skills, systemd service, crontab.
.taskfiles/	Taskfile-uri modulare: bootstrap, talos, longhorn.
Taskfile.yaml	Entry point principal: task reconcile, task dr:*, task longhorn:*, task talos:*
.mise.toml	Gestioneaza toate versiunile de unelte CLI (kubect!, flux, talosctl, sops etc.).
.sops.yaml	Regulile de criptare SOPS — ce fisiere se cripteaza si cu ce cheie age.
age.key	Cheia privata age pentru SOPS. NICIODATA commit. Back up obligatoriu!
bootstrap/helmfile.yaml	Ordinea de instalare Cilium→CoreDNS→cert-manager→Flux la bootstrap.

2. Tooling si Configuratie Globala

► .mise.toml — Versiunile uneltelor

mise este un manager de versiuni universal (inlocuieste asdf/nvm/pyenv). Toate uneltele sunt definite la versiune exacta — garanteaza reproducibilitate intre masini.

Unealta + versiune	Rol
<code>python 3.14.5</code>	Python — pentru scripturi interne (dr:apply-talos-configs, gen PDF etc.)
<code>talhelper 3.1.10</code>	Genereaza configuratii Talos din talconfig.yaml (abstractizare YAML→MachineConfig)
<code>talosctl 1.13.3</code>	CLI pentru nodurile Talos (apply-config, upgrade, reset, logs)
<code>kubect1 1.36.1</code>	CLI Kubernetes standard
<code>flux 2.8.8</code>	CLI Flux — reconcile, check status, get sources/kustomizations
<code>helm 4.2.0</code>	Package manager Kubernetes — folosit de helmfile la bootstrap
<code>helmfile 1.5.2</code>	Orchestrare multi-Helm cu dependente (Cilium → CoreDNS → Flux)
<code>sops 3.13.1</code>	Criptare/decriptare secrete YAML cu cheia age
<code>age 1.3.1</code>	Algoritmul de criptare folosit de SOPS (modern, simplu)
<code>kustomize 5.7.1</code>	Overlay-uri Kubernetes — Flux foloseste kustomize intern
<code>yq 4.53.3</code>	Procesare YAML in CLI (folosit in scripturi Taskfile)
<code>jq 1.7.1</code>	Procesare JSON in CLI
<code>kubeconform 0.8.0</code>	Validare schema YAML Kubernetes in CI/CD
<code>cilium-cli 0.19.4</code>	CLI pentru diagnosticare Cilium (connectivity test, status)
<code>gh 2.93.0</code>	GitHub CLI — PRs, issues, workflow runs din terminal
<code>cloudflared 2026.5.2</code>	Cloudflare Tunnel daemon (accesibil din browser fara port-forwarding)

► .sops.yaml — Regulile de criptare

SOPS cripteaza automat secretele cu cheia age. Fisierele .sops.yaml din repo NU pot fi citite fara cheia privata din age.key.

Regula	Explicatie
<code>talos/*.sops.yaml</code>	<code>mac_only_encrypted: true</code> — cripteaza doar valorile, nu cheile YAML
<code>kubernetes/*.sops.yaml</code>	<code>encrypted_regex: ^([data stringData])\$</code> — cripteaza doar datele secretelor K8s
<code>age: age169tkyzj...</code>	Cheia PUBLICA age (ok in repo). Cheia privata e in age.key (gitignored)

► Taskfile.yaml — Entry points principale

Comanda	Ce face
<code>task reconcile</code>	Forțeaza Flux sa preia modificarile din Git imediat
<code>task dr:create-vms</code>	Creeaza 3 VMs Talos pe Proxmox via Terraform (DR)
<code>task dr:apply-talos-configs</code>	Scan subnet, detecteaza Talos maintenance mode, aplica configs
<code>task dr:verify</code>	Rulare completa dr-verify.sh — verifica DR sfarsit-la-sfarsit
<code>task dr:destroy-vms</code>	Distruge VMs DR pe Proxmox dupa test
<code>task bootstrap:talos</code>	Bootstrap cluster Talos de la zero
<code>task bootstrap:apps</code>	Instaleaza componentele de baza via helmfile
<code>task longhorn:restore</code>	Restaureaza volumele Longhorn din backup S3
<code>task longhorn:opgbornestatesjellyfin</code>	Re-triggereaza refresh imagini Jellyfin dupa DR
	Afiseaza statusul volumelor + PVC-urilor + HelmRelease-urilor

```
task talos:upgrade-node IP=x
```

task talos:upgrade-k8s

Upgrade Talos pe un singur nod

Upgrade versiunea Kubernetes pe intregul cluster

3. VPS Layer — Ansible

Directorul vps/ gestioneaza VPS-ul Oracle Cloud prin Ansible (configurare idempotentă) și Terraform (provisionare fallback pe Hetzner pentru DR).

► vps/Makefile — Interfața principală

Comanda	Ce face
<code>make setup</code>	Rulează site.yml — deploy complet idempotent a tuturor serviciilor Docker
<code>make health-check</code>	Verificare post-deploy: containere running, endpoint-uri accesibile
<code>make update</code>	Doar update-uri OS (apt upgrade), fara redeployment servicii
<code>make check</code>	Dry-run: simulează ce ar schimba fara sa aplice nimic
<code>make vault-edit</code>	Deschide editorul criptat pentru secretele Ansible (ansible-vault)
<code>make dr-full</code>	Disaster Recovery complet: dr-preflight + terraform-apply + setup
<code>make terraform-apply</code>	Provisionează VPS nou pe Hetzner + updatează inventory cu noul IP
<code>make garage-extract-creds</code>	Extrage credentiale S3 Garage noi + update vault (între faze DR)
<code>make authentik-backup</code>	Backup baza de date Authentik → /srv/backups/authentik/

► Playbook-uri Ansible principale

- site.yml — orchestrator: include toate rolurile în ordine corectă
- traefik-setup.yml — reverse proxy cu ACME (Let's Encrypt) via Cloudflare DNS
- authentik-setup.yml — SSO identity provider (PostgreSQL + Redis + Worker + Server)
- garage-setup.yml — storage S3 self-hosted (backup target pentru Longhorn K8s)
- docker-setup.yml — instalare Docker Engine + Compose + configurare daemon
- health-check.yml — verifică ca toate containerele sunt Running după deploy
- cleanup.yml — șterge resurse Docker neutilizate (imagini vechi, volume orfane)

► Vault Ansible — Secretele VPS-ului

Fisierul inventories/production/group_vars/all/vault.yml este criptat cu ansible-vault. Parola e în .vault_pass (gitignored). Conține:

Variabila	Utilizare
<code>cloudflare_api_token</code>	Token API Cloudflare — Traefik pentru DNS challenge ACME TLS
<code>traefik_dashboard_credentials</code>	htpasswd pentru accesul la dashboard-ul Traefik
<code>vault_tailscale_auth_key</code>	Cheia de auth Tailscale — VPN mesh între VPS și cluster K8s
<code>pihole_webpassword</code>	Parola Pi-hole DNS ad-blocker
<code>garage_rpc_secret</code>	Secretul intern Garage (comunicare între noduri S3)
<code>garage_admin_token</code>	Token admin pentru Garage API (creare buckets, keys)
<code>authentik_pg_pass</code>	Parola PostgreSQL pentru Authentik SSO
<code>authentik_secret_key</code>	Cheia secretă Authentik (sesiuni, tokens, CSRF)
<code>vault_joplin_db_password</code>	Parola baza de date Joplin (aplicație note)

► Terraform VPS — Hetzner (DR fallback)

vps/terraform/ provisionează un server nou pe Hetzner Cloud pentru DR. Când VPS-ul Oracle pica, make dr-full creează automat un server identic și redeploya tot stack-ul Docker.

- Tipul serverului, locația, SSH key, firewall — toate configurabile în terraform.tfvars
- După terraform apply, Makefile actualizează automat inventory Ansible cu noul IP

- make setup ruleaza imediat dupa — toate serviciile Docker sunt reproductibile 100%
- Garage S3 primeste credentiale noi la fiecare instanta — make garage-extract-creds actualizeaza vault

4. Talos Linux — OS-ul Kubernetes

Talos este un OS imutabil proiectat exclusiv pentru Kubernetes. Nu exista SSH, nu exista shell interactiv — totul se configureaza declarativ prin YAML. talhelper simplifica generarea configuratiilor.

► talos/talconfig.yaml — Configuratia clusterului

Camp talconfig.yaml	Explicatie
<code>clusterName: kubernetes</code>	Numele clusterului — apare in kubeconfig si in Talos API
<code>endpoint: 10.57.57.88:6443</code>	VIP (Virtual IP) — adresa API server-ului K8s. Failover automat
<code>clusterPodNets: 10.42.0.0/16</code>	Reteaua interna a Pod-urilor (gestionata de Cilium)
<code>clusterSvcNets: 10.43.0.0/16</code>	Reteaua serviciilor K8s (ClusterIP)
<code>cniConfig: name: none</code>	FARA CNI built-in — Talos nu instaleaza nimic, Cilium vine separat
<code>talosImageURL: factory.talos.dev/installer/8d37f..</code>	Imagine factory.talos.dev cu extensii precompilate: intel-ucode, i915 GPU driver
<code>allowSchedulingOnControlPlanes</code>	In patches/machine-kubelet.yaml: true — toate 3 noduri = si worker

Cele 3 noduri controlplane

```
kubernetes-controlplane-1: IP=10.57.57.80  MAC=bc:24:11:a7:ba:13  GPU=Intel Arc (i9-13900HK)
kubernetes-controlplane-2: IP=10.57.57.82  MAC=bc:24:11:a5:4b:9e
kubernetes-controlplane-3: IP=10.57.57.84  MAC=bc:24:11:0e:cd:ab
VIP (Kubernetes API):      IP=10.57.57.88  → failover automat intre noduri
```

Toate 3 au: installDisk=/dev/sda, secureboot=false, dhcp=false (IP static)
Gateway: 10.57.57.1 (routerul/pfSense-ul local)

► talos/talenv.yaml — Versiunile

```
talosVersion:      v1.13.3  # Renovate Bot actualizeaza automat, deschide PR cu versiunea noua
kubernetesVersion: v1.36.1  # Renovate Bot actualizeaza automat
```

► talos/patches/ — Modificarile fata de default

Fisier patch	Ce modifica
<code>machine-kubelet.yaml</code>	<code>allowSchedulingOnControlPlanes: true</code> — nodurile = si controlplane si worker
<code>machine-network.yaml</code>	DNS servers, extraHostEntries — configurare retea avansata
<code>machine-longhorn.yaml</code>	Monteaza /var/lib/longhorn pe disk — necesar pentru Longhorn storage
<code>machine-longhorn-labels.yaml</code>	Label node.longhorn.io/create-default-disk: config — Longhorn recunoaste nodul
<code>machine-files.yaml</code>	Fisiere injectate in OS: udev rules pentru GPU, configuratii sistem
<code>machine-sysctls.yaml</code>	Parametri kernel: fs.inotify.max_user_watches, net.core.* etc.
<code>machine-time.yaml</code>	NTP servers (pool.ntp.org), timezone Europe/Bucharest
<code>admission-controller-patch.yaml</code>	Configurare K8s admission controllers (securitate Pod-uri)
<code>cluster.yaml</code>	etcd backup config, scheduler extra args, API server flags

5. Terraform — Disaster Recovery VMs pe Proxmox

talos/terraform/ creeaza 3 masini virtuale pe Proxmox pentru testarea/executarea DR-ului. Foloseste provider-ul `bpg/proxmox` (cel mai matur provider Proxmox pentru Terraform).

► talos/terraform/main.tf — Ce se creeaza

Resursa	Explicatie
<code>proxmox_virtual_environment_download_iso</code>	Descarca ISO Talos de pe <code>factory.talos.dev</code> direct in storage Proxmox.
<code>proxmox_virtual_environment_vm</code>	Idempotent (nu re-descarca daca exista)
<code>proxmox_virtual_environment_vm</code>	3 VMs distribuite pe nodurile Proxmox (<code>px-0</code> , <code>px-1</code> , <code>px-2</code>). CPU=host, 8GB RAM, 500GB disk
<code>network_device.mac_address</code> = <code>var.node_mac[i]</code>	MAC-uri FIXE = identice cu productia! DHCP da aceleasi IP-uri. talconfig nu necesita modificare.
<code>efi_disk + boot_order: ide2-scsi0</code>	Boot din ISO (Talos maintenance mode), ulterior din disk. EFI disk necesar pentru UEFI boot.
<code>lifecycle.ignore_changes</code> = <code>[cdrom] [talos, k8s-dr]</code>	Terraform nu distruge VM-ul cand ISO este scos dupa instalare Talos VM-urile DR sunt marcate in Proxmox pentru identificare usoara

► talos/terraform/variables.tf — Parametrii configurabili

Variabila	Valoare / Rol
<code>proxmox_url</code>	URL Proxmox API: <code>https://10.57.57.254:8006</code>
<code>proxmox_nodes</code>	Noduri Proxmox: <code>["px-0", "px-1", "px-2"]</code>
<code>vm_cores</code> / <code>vm_memory_mb</code> / <code>vm_vdisks_gb</code>	4 vCPU, 8192 MB RAM, 500 GB disk per VM
<code>vm_vdisks_gb</code>	VMID-urile DR incep de la 810 (productia foloseste 800-805)
<code>node_mac</code>	MAC-uri identice cu productia — ESSENTIAL pentru a nu patcha talconfig
<code>talos_version</code>	v1.13.3 — trebuie sa corespunda cu <code>talenv.yaml</code>
<code>talos_image_id</code>	Hash-ul imaginii <code>factory.talos.dev</code> (extensiile precompilate: intel-ucode, i915)
<code>disk_storage</code>	cluster-storage — storage-ul Proxmox pentru disk-urile VM-urilor

De ce MAC-uri fixe identice cu productia?

Talos identifica interfata de retea via `deviceSelector.hardwareAddr` in `talconfig.yaml`.

Cu acelasi MAC, DHCP-ul local da acelasi IP → IP-urile statice se configureaza corect.

Rezultat: nu trebuie modificat `talconfig.yaml` pentru DR → procedura mai simpla si fiabila.

task `dr:patch-talconfig` exista ca fallback pentru hardware diferit (non-prod).

6. Bootstrap — Prima Pornire a Clusterului

bootstrap/helmfile.yaml instalează componentele de bază în ordine strict definită. Fiecare componentă are `needs:` — nu se instalează dacă precedentă a eșuat.

► bootstrap/helmfile.yaml — Ordinea de instalare

1. cilium (kube-system) v1.19.4 — CNI; fără el nici un pod nu pornește
needs: nimic — primul întotdeauna
2. coredns (kube-system) v1.46.0 — DNS intern cluster
needs: cilium — necesită rețea funcțională
3. spegel (kube-system) v0.7.1 — P2P image cache între noduri
needs: coredns — necesită rezoluție DNS pentru discovery
4. cert-manager (cert-manager) v1.20.2 — TLS certificates (webhooks Flux)
needs: spegel — beneficiază de cache imagini
5. flux-operator (flux-system) v0.50.0 — Operatorul Flux
needs: cert-manager — webhooks Flux au nevoie de TLS
6. flux-instance (flux-system) v0.50.0 — Instanța Flux (citește repo-ul Git)
needs: flux-operator — operatorul trebuie să existe primul

După pasul 6: Flux preia controlul. Tot restul (Longhorn, apps etc.) vine din Git.

► task bootstrap:talos — Bootstrap cluster Talos

1. [-f talsecret.sops.yaml] || talhelper gensecret | sops --encrypt → talsecret.sops.yaml
(o singură dată — generează CA, etcd key, bootstrap token și le criptează cu SOPS)
2. talhelper genconfig
(generează clusterconfig/*.yaml din talconfig.yaml + talsecret.sops.yaml)
3. talhelper gencommand apply --insecure | bash
(aplică configurația la noduri în maintenance mode — înainte de reboot)
4. until talhelper gencommand bootstrap | bash
(initializează etcd cluster — se repetă până reușește)
5. talhelper gencommand kubeconfig --extra-flags="ROOT_DIR --force" | bash
(generează kubeconfig local pentru kubectl)

7. Flux GitOps

Flux urmărește continuu repo-ul Git (branch: main) și aplică automat orice modificare. Dacă modifici un HelmRelease și faci push, Flux detectează în <1 minut și aplică în cluster.

► Componente Flux și rolul lor

Componenta	Rol
<code>source-controller</code>	Descarcă conținut din GitHub (polling 1 min sau webhook instant)
<code>kustomize-controller</code>	Aplică manifeste K8s (kustomization.yaml → kubectl apply)
<code>helm-controller</code>	Gestionează HelmRelease-urile (install/upgrade/rollback Helm charts)
<code>notification-controller</code>	Webhook GitHub pentru trigger instant la push

► flux-instance/app/helm/values.yaml — Optimizarile Flux

Configurare	Efect
<code>sync.url</code>	<code>https://github.com/meroxdotdev/infrastructure.git</code>
<code>sync.path:</code>	Directorul de start — Flux citește de aici kustomization-urile
<code>sync.ref: refs/heads/main</code>	Branch-ul urmărit — doar main este sursa de adevăr
<code>--concurrent=10 (kustomize)</code>	Procesare paralelă a 10 kustomization-uri simultan
<code>emptyDir: medium: Memory</code>	Build-uri kustomize în memorie (mai rapid, nu scrie pe disk)
<code>--helm-cache-max-size=10</code>	Cache 10 chart-uri Helm în memorie (reduce download-uri repetate)
<code>--sops-age-secret=sops-age</code>	Flux decriptează automat secretele SOPS cu cheia age din K8s secret
<code>OOMWatch=true, threshold=95%</code>	Dacă helm-controller consumă >95% RAM, reporneste controlat
<code>memory: limits: 1Gi</code>	Limita memorie per controller Flux
<code>CancelHealthCheckOnNewRevision</code>	Nu așteaptă health check vechi dacă vine o versiune nouă

► Structura kustomization-urilor (ierarhia Flux)

```
kubernetes/flux/cluster/      ← punct de start Flux (sync.path)
├─ apps/kube-system/kustomization.yaml ← namespace kube-system
├─ apps/storage/kustomization.yaml    ← Longhorn, restore-pvs
├─ apps/cert-manager/kustomization.yaml ← cert-manager + ClusterIssuer
├─ apps/observability/kustomization.yaml ← Prometheus, Grafana, Loki
├─ apps/default/kustomization.yaml    ← toate aplicațiile (media, n8n etc.)
│   └─ jellyfin/ks.yaml (Kustomization Flux)
│       └─ dependsOn: longhorn ← Jellyfin așteaptă Longhorn
│           └─ app/kustomization.yaml
│               └─ helmrelease.yaml ← configurația aplicației
│                   └─ pvc.yaml      ← storage-ul persistent
```

8. Networking — Cilium (CNI + Gateway API)

Cilium este CNI-ul bazat pe eBPF. Inlocuieste complet kube-proxy si adauga Gateway API, L2 LoadBalancer IPs, politici de retea avansate. Fara overlay/tunnel — rutare nativa Layer 3.

► cilium/app/helm/values.yaml — Configuratii cheie

Configurare	Explicatie
<code>kubeProxyReplacement: true</code>	Cilium inlocuieste kube-proxy complet (eBPF, mai rapid)
<code>k8sServiceHost: 127.0.0.1:7445</code>	API Server accesat local (Talos LocalAPI) — evita traficul prin VIP
<code>routingMode: native</code>	Rutare nativa Layer 3 — fara VXLAN/Geneve overlay (maxim performanta)
<code>autoDirectNodeRoutes: true</code>	Rutare directa intre noduri — pod pe nodul A ajunge direct la pod pe nodul B
<code>bpf.masquerade: true</code>	SNAT implementat in eBPF (mai rapid decat iptables)
<code>bpf.hostLegacyRouting: true</code>	Fix pentru Talos (issue siderolabs/talos#10002)
<code>ipam.mode: kubernetes</code>	IPAM gestionat de K8s (cidruri din clusterPodNets 10.42.0.0/16)
<code>l2announcements.enabled: true</code>	Cilium anunta IP-urile LoadBalancer via ARP (ca MetalLB, fara BGP)
<code>loadBalancer.algorithm: maglev</code>	Consistent hashing pentru distributia traficului LB
<code>loadBalancer.mode: dsr</code>	Direct Server Return — raspunsul merge direct la client, nu prin LB
<code>gatewayAPI.enabled: true</code>	Gateway API activat — inlocuieste Ingress cu Gateway/HTTPRoute
<code>hubble.enabled: false</code>	Observabilitate Cilium dezactivata (economie resurse)
<code>socketLB.hostNamespaceOnly</code>	Socket-based LB doar in host namespace (compatibilitate Talos)

► Gateway API — doua Gateway-uri

Element	Rol
<code>Gateway: internal</code>	LAN + Tailscale. *.k8s.merlox.dev. Toate aplicatiile interne.
<code>Gateway: external</code>	Internet public prin Cloudflare. Doar webhook-uri si servicii publice.
<code>sectionName: https</code>	Fiecare HTTPRoute specifica pe care Gateway se ruteaza (internal/external)
<code>certificate.yaml</code>	cert-manager genereaza certificat wildcard *.k8s.merlox.dev via DNS challenge
<code>parentRef: internal</code>	Aplicatia este accesibila doar din LAN/Tailscale (nu din internet)
<code>parentRef: external</code>	Aplicatia este accesibila public (ex: n8n-webhook pentru automatizari)

9. Storage — Longhorn

Longhorn este distributed block storage pentru Kubernetes. Fiecare volum are 3 replici pe cele 3 noduri. Backup automat zilnic in S3 (Garage self-hosted pe VPS Oracle).

► longhorn/app/helmrelease.yaml — Setările importante

Configurare	Explicatie
<code>defaultReplicaCount: 3</code>	3 replici per volum — supravietuieste pierderea unui nod
<code>defaultDataLocality: best-effort</code>	Prefera o replica pe nodul care ruleaza pod-ul (latenta redusa)
<code>backupTarget: s3://longhorn@...</code>	Target S3 pentru backup — Garage local pe VPS Oracle
<code>backupTargetCredentialSecret</code>	Secret K8s cu credentialele S3 (HMAC key din Garage)
<code>backupstorePollInterval: 300</code>	Verifica S3 din 5 in 5 minute pentru noi backup-uri
<code>nodeDownPodDeletionPolicy</code>	Daca un nod pica, sterge pod-urile → relocare rapida pe alte noduri
<code>orphanAutoDeletion: true</code>	Sterge automat volumele orfane (PV fara PVC)
	Alerta daca spatiu liber scade sub 1%
<code>replicaAutoBalance: true</code>	Rebalanseaza replicile automat dupa restart noduri
<code>1 defaultClass: true</code>	Longhorn este StorageClass-ul default — PVCs fara storageClass merg pe Longhorn

► longhorn/recurringjob/recurringjob.yaml — Backup automat

```
name: backup-homelab-k8s
cron: "00 02 * * *" # in fiecare zi la 02:00 noaptea
task: backup # backup complet (nu snapshot local)
groups: [default] # afecteaza volumele cu label recurring-job-group.longhorn.io/default: enabled
retain: 3 # pastreaza ultimele 3 backup-uri per volum
```

Volumele include (au label backup + backup-volume: name-restored):

jellyfin, jellyseerr, prowlarr, qbittorrent, radarr, sonarr,
n8n, grafana, prometheus, loki-0

Volumele cu backup DAR fara restore la DR (volum Longhorn cu nume dinamic):

jellyfin-cache, sonarr-cache, radarr-cache, jellyseerr-cache
→ dupa DR: task longhorn:post-restore-jellyfin (reimagineaza in ~10 min)

► restore-pvs/pvs.yaml — PV-urile pentru DR restore

La DR, task longhorn:restore creeaza volume Longhorn din backup S3, apoi pvs.yaml leaga PV → PVC cu claimRef explicit.

PersistentVolume	Legat la
<code>jellyfin-restored-pv</code>	PVC jellyfin (10Gi) — configuratie, baza de date, plugin-uri Jellyfin
<code>jellyseerr-restored-pv</code>	PVC jellyseerr (2Gi) — configuratie si DB Jellyseerr
<code>radarr-restored-pv</code>	PVC radarr (5Gi) — baza de date filme (SQLite)
<code>sonarr-restored-pv</code>	PVC sonarr (5Gi) — baza de date seriale (SQLite)
<code>prowlarr-restored-pv</code>	PVC prowlarr (2Gi) — configuratie indexori torrent
<code>qbittorrent-restored-pv</code>	PVC qbittorrent (5Gi) — configuratie si seed-uri qBittorrent
<code>grafana-restored-pv</code>	PVC grafana (10Gi) — dashboard-uri, datasource-uri, users
<code>loki-restored-pv</code>	PVC storage-loki-0 (20Gi) — log-uri istorice
<code>prometheus-restored-pv</code>	PVC prometheus (15Gi) — metrice istorice
<code>n8n-restored-pv</code>	PVC n8n (5Gi) — workflow-uri n8n (dezactivat implicit)

10. Certificate Management — cert-manager

cert-manager gestionează automat certificatele TLS. Folosește Lets Encrypt cu DNS-01 challenge via Cloudflare API — funcționează și pentru servicii complet interne (nu necesită port 80/443 deschis).

► ClusterIssuer letsencrypt-production

Camp	Explicatie
<code>acme.server</code>	<code>https://acme-v02.api.letsencrypt.org</code> — LE productie (nu staging)
<code>profile: shortlived</code>	Certificate cu validitate mai mica (~30 zile) — mai sigure, rotatie frecventa
<code>dns01.cloudflare</code>	Challenge DNS via Cloudflare API — cert-manager creează TXT record temporar
<code>apiTokenSecretRef:</code>	Token Cloudflare din secretul SOPS-criptat
<code>dnsZones: \${SECRET_DOMAIN}</code>	Doar pentru domeniul merlox.dev (variabila substituita de Flux)

Flux de obtinere certificat (DNS-01 challenge):

1. cert-manager vede Gateway/HTTPRoute cu `tls.certificateRef` → creează Certificate CRD
2. Certificate CRD → ACME challenge DNS-01 → cert-manager creează TXT record in Cloudflare
3. Lets Encrypt verifică TXT record → emite certificat → cert-manager pune in Secret K8s
4. Cilium Gateway citește Secret-ul → servește HTTPS cu certificatul valid

Avantaj: funcționează pentru `*.k8s.merlox.dev` chiar dacă serviciile nu sunt accesibile public

11. Aplicații — Namespace default

Toate aplicațiile folosesc chart-ul `bjw-s/app-template` via `OCIRepository` (`HelmRelease`). Pattern comun: `helmrelease.yaml` + `pvc.yaml`. PVC-urile sunt pe Longhorn, media este pe NAS via NFS.

► Jellyfin — Media Server (HTPC)

Configurare	Rol
<code>image: ghcr.io/jellyfin/jellyfin:10.11.10</code>	Server media: filme, seriale, muzica
<code>JELLYFIN_PublishedServerUrl: https://media.merox.dev</code>	Acces GPU Intel Arc (i9-13900HK) — hardware transcoding QSV
<code>readOnlyRootFilesystem: true</code>	all= <code>https://media.merox.dev</code> — URL public pentru clienți externi
<code>supplementalGroups: [44]</code>	Filesystem root read-only — securitate sporită
<code>PVC jellyfin (10Gi) → /config</code>	Grup video — necesar pentru <code>/dev/dri</code> (GPU render node)
<code>PVC jellyfin-cache (10Gi) → /cache</code>	Configurație, baza de date, plugin-uri
<code>/cifs/mount/mediaserver/ → /media</code>	Postere, backdrops, thumbs (regenerabil post-DR)
<code>emptyDir → /cache</code>	Fisierele media de pe NAS Synology (read-only)
<code>emptyDir → /config/log + /tmp</code>	Transcode cache — ephemeral, se pierde la restart
	Loguri și temp — nu persist (readOnly filesystem workaround)

► Stack media: Sonarr + Radarr + Prowlarr + qBittorrent

Componenta	Rol
Sonarr	Gestionar seriale TV — monitorizează, descarcă automat, redenumeste episoadele
Radarr	Gestionar filme — același concept ca Sonarr dar pentru filme
Prowlarr	Agregator indexori torrent — o singură configurație pentru Sonarr+Radarr
qBittorrent	Client torrent — rulează în interiorul containerului Gluetun (VPN mandatory)
Gluetun sidcar	Container VPN (Surfshark WireGuard) — qBittorrent nu are acces la internet fără VPN
NFS mount	Toate au acces la <code>/media</code> pe NAS — fișierele se scriu direct acolo
LB IP qBit	LoadBalancer pe portul 50413 TCP (<code>lbipam.cilium.io/ips</code> pentru IP fix)
Fiecare are PVC	2-5Gi Longhorn pentru config + SQLite DB — restaurate din backup la DR

► Jellyseerr — Gestionar request-uri media

Interfață pentru utilizatori să ceară filme/seriale. Trimite automat la Radarr/Sonarr pentru descărcare. Autentificare via contul Jellyfin.

► n8n — Automatizare (no-code)

Configurare	Rol
<code>image: ghcr.io/n8n-io/n8n:2.25.5</code>	Platforma workflow automation — similar Zapier dar self-hosted
<code>route app: n8n.k8s.merox.dev</code>	Interfață — accesibilă intern prin Tailscale/LAN
<code>route webhooks: n8n.k8s.merox.dev</code>	Webhook-uri accesibile public (Gateway external)
<code>DB_SQLITE_VACUUM_ON_STARTUP: true</code>	Cheie pentru criptarea credențialelor stocate în n8n
<code>PVC n8n (5Gi) → /home/node/.n8n</code>	Optimizare SQLite la fiecare pornire
	Workflow-uri, execuții, credențiale — restaurat din backup la DR

► Homepage — Dashboard

Dashboard de start configurat via ConfigMap (resources/). Serveste statusul serviciilor, widget-uri (vreme, calendar, metrice). Nu necesita PVC — configuratia este in Git.

► Authentik Outpost — SSO Proxy K8s

Proxy-ul Authentik care ruleaza in K8s. Se conecteaza la instanta Authentik de pe VPS (sso.meron.dev) si protejeaza serviciile K8s cu SSO (Google-only login).

Configurare	Rol
<code>AUTHENTIK_HOST:</code>	Instanta principala Authentik (pe VPS)
<code>HTTP_PROXY_BROWSER</code>	URL-ul prin care browserul utilizatorului acceseaza Authentik
<code>envFrom: authentik-outpost-secret</code>	Token de autentificare outpost (din 1Password via ESO)
<code>port 9000</code>	Port intern outpost — HTTPRoute ruteaza traficul prin el

12. Observability — Monitorizare

Stack complet: metrice (Prometheus + Grafana), log-uri (Loki + Promtail), alertare (Alertmanager → Telegram). Netdata adauga metrice real-time la 1 secunda granularitate.

► kube-prometheus-stack

- Prometheus (15Gi PVC) — colecteaza metrice de la toate pod-urile K8s via ServiceMonitor CRDs
- Alertmanager — gestioneaza alertele, trimite notificari (Telegram, email)
- Node Exporter — metrice hardware de la fiecare nod (CPU, RAM, disk, retea)
- kube-state-metrics — starea resurselor K8s (deployments, pods, PVCs etc.)
- scrapeconfig.yaml — configuratii de scraping pentru servicii externe clusterului
- alertmanagerconfig.yaml — rutare alerte catre receptori (Telegram bot configurabil)

► Grafana (10Gi PVC)

- Vizualizare metrice Prometheus + log-uri Loki in dashboard-uri
- PVC grafana (10Gi) — dashboard-uri salvate, datasource-uri configurate (restaurate la DR)
- Autentificare via Authentik SSO (outpost-ul K8s)
- externalsecret.yaml — credentiale admin din 1Password via External Secrets Operator

► Loki + Promtail (20Gi PVC)

- Loki — baza de date pentru log-uri (similar Elasticsearch dar mai usor, bazat pe labels)
- PVC loki (20Gi) — log-uri istorice persistente (restaurate la DR)
- Promtail — DaemonSet pe fiecare nod, colecteaza log-uri din containere + /var/log
- lokirule.yaml — aplicatiile media (Jellyfin, Radarr etc.) au reguli de filtrare/parsare loguri
- patches/statefulset-sidecar-tmpdir.yaml — fix pentru readOnlyRootFilesystem pe Loki

► Netdata

Configurare	Rol
<code>parent + child claiming</code>	Ambele (K8s nodes + VPS) vizibile in acelasi room Netdata Cloud
<code>rooms: 536268ca-...</code>	ID-ul camerei — toti senzorii vizibili intr-un singur loc
<code>valuesFrom: Secret</code>	Claiming token din secretul netdata-secret (criptat SOPS)
<code>storageclass: longhorn</code>	Date Netdata pe Longhorn (5Gi DB + 1Gi alarms + 1Gi k8s-state)
<code>image.tag: edge</code>	Versiunea edge — cele mai recente imbunatatiri

13. Agenti AI — OpenClaw

OpenClaw este un framework de agenti AI bazat pe Claude Sonnet. Ruleaza pe VPS ca serviciu systemd. Accesibil prin Telegram (bot) sau gateway HTTP intern (Tailscale Serve, nu expus public).

► agent/openclaw.json — Configuratia principala

Configurare	Rol
<code>model.primary: claude-sonnet-4-6</code>	Model Anthropic implicit pentru toti agentii
<code>channels.telegram</code>	Bot Telegram — doar userul tau poate trimite DM-uri (allowlist)
<code>gateway.bind: loopback</code>	Gateway asculta doar pe localhost — expus prin Tailscale Serve
<code>gateway.auth.mode: token</code>	Autentificare prin token pentru accesul la gateway HTTP
<code>gateway.tailscale.mode: serve</code>	Tailscale Serve mapeaza HTTPS pe .ts.net la gateway-ul local
<code>dmPolicy: allowlist</code>	Securitate — bot-ul ignora mesajele de la oricine altcineva
<code>allowFrom: [TELEGRAM_USER_ID]</code>	Doar ID-ul tau Telegram poate interactiona cu botul

► Workspaces — Cei 8 agenti

Workspace	Rol
<code>news/</code>	Briefing zilnic de stiri — agregare RSS, sinteza AI, postat pe dashboard si Telegram
<code>infra/</code>	Gestionar infrastructura — kubectl, flux, verificari cluster, alertare Telegram
<code>dashboard/</code>	Actualizare dashboard /srv/dashboard — genereaza index.html nocturn
<code>costs/</code>	Monitorizare costuri cloud — Oracle, Hetzner, analiza cheltuieli, rapoarte
<code>blog/</code>	Asistent scriere blog merox.dev (Astro static site) — draft-uri, publicare
<code>design/</code>	Asistent design UI/UX pentru proiecte
<code>orchestrator/</code>	Meta-agent — coordoneaza ceilalti agenti, decide care raspunde la un request complex
<code>renovate/</code>	Analizeaza PR-urile Renovate Bot — aproba/respinge update-uri de versiuni
<code>repo/</code>	Asistent pentru repo-ul de infrastructura — documentatie, review cod, debug

► agent/scripts/ — Infrastructura de rulare

Fisier	Rol
<code>openclaw-gateway.service</code>	Systemd USER service (ruleaza ca user openclaw, nu root)
<code>openclaw-crontab</code>	Cron jobs: stiri dimineata, dashboard noaptea, update-infra la fiecare ora
<code>openclaw-fix-perms</code>	Script sudoers: ajusteaza permisiunile fisierelor dupa restart
<code>sudoers-openclaw</code>	Regulile sudo pentru userul openclaw (kubectl, docker citire etc.)

► agent/dashboard/scripts/ — Scripturile de update dashboard

Script	Ce face
<code>update-infra.sh</code>	kubectl get nodes/pods/pvc → JSON → /srv/dashboard/data/infra.json
<code>update-weather.sh</code>	OpenWeatherMap API → JSON → /srv/dashboard/data/weather.json
<code>update-backup.sh</code>	Verifica backup-uri Longhorn in Garage S3 → status.json
<code>update-network.sh</code>	Ping targets, Tailscale status → network.json
<code>tg-notify.sh</code>	Trimite notificari Telegram (alertare, heartbeat, anomalii)
<code>self-healing.sh</code>	Verifica servicii critice, reporneste cele cazute automat

14. Secrets Management — SOPS + age

Toate secretele din repo sunt criptate cu SOPS folosind algoritmul age. Cheia PUBLICA age este in .sops.yaml (poate fi in repo). Cheia PRIVATA este in age.key (gitignored, TREBUIE backup-ata).

► Cum functioneaza criptarea

```
# Criptare (necesita cheia publica din .sops.yaml):
sops --encrypt secrets.yaml > secrets.sops.yaml

# Decriptare (necesita age.key in $SOPS_AGE_KEY_FILE):
sops --decrypt secrets.sops.yaml

# Editare directa in editor (salveaza criptat automat):
sops secrets.sops.yaml

# Flux decripteaza automat toate fisierele .sops.yaml din repo:
# kustomize-controller are --sops-age-secret=sops-age
# Secret K8s "sops-age" in flux-system contine continutul lui age.key
# Oricum Kustomization cu decryption: provider: sops beneficiaza de decriptare automata
```

► Tipuri de secrete in repo

Fisier	Continut
talos/talsecret.sops.yaml	Secretele Talos: cluster CA, etcd encryption key, bootstrap token
kubernetes/*/secret.sops.yaml	Secrete K8s: API keys, parole servicii, token-uri externe
qbittorrent/patches/secrets/secrets.sops.yaml	Credentiale Surfshark VPN (WireGuard private key, address)
vps/.vault_pass	Credentiale S3 Garage (access key, secret key) pentru backup
Ansible vault.yml	Parola Ansible Vault (gitignored, mecanism separat de criptare)
Ansible vault.yml	Secrete VPS: Cloudflare token, Tailscale key, parole servicii Docker

► Renovate Bot — Actualizare automata versiuni

.renovaterc.json5 configureaza Renovate Bot (GitHub App) sa scaneze toate fisierele dupa versiuni si sa deschida PR-uri automate:

- Imagini Docker (tag: in HelmRelease) — detecteaza versiuni noi pe ghcr.io, docker.io
- Helm charts (version: in HelmRelease) — versiuni noi din OCI registries
- talosVersion/kubernetesVersion din talenv.yaml — datasource: docker
- Versiunile din .mise.toml — unelte CLI (kubect!, flux, talosctl etc.)
- SHA digest pinning — imaginile Docker sunt fixate la SHA256 exact (nu doar tag)
- Agentul renovate/ din OpenClaw revizuieste PR-urile Renovate inainte de merge

15. Disaster Recovery — Procedura Completa

DR-ul este testat end-to-end (06.06.2026). Exista doua scenarii: pierderea VPS-ului Oracle si pierderea clusterului K8s. Ambele sunt automatizate.

► Scenariul 1 — VPS Oracle pica (Ansible + Hetzner)

```
cd /srv/kubernetes/infrastructure/vps

make dr-preflight      # verifica prereqs: terraform state, vault pass, SSH key, deps
make dr-full          # = terraform-apply + wait SSH + setup (~15 min total)

# terraform-apply → Hetzner VPS nou → IP nou → Ansible inventory auto-update
# setup → ansible-playbook site.yml → deploy complet Docker stack idempotent

make garage-extract-creds # extrage credentiale S3 Garage noi → update vault Ansible
make dr-verify-phase1     # verifica: containere Running, Traefik HTTPS, Authentik OK
make dr-verify-phase3     # verifica: OpenClaw agents, dashboard, Telegram bot
```

► Scenariul 2 — Cluster K8s pica (Talos + Flux + Longhorn)

```
PASUL 1: Provizionare VMs Talos pe Proxmox
task dr:create-vms
# Terraform: 3 VMs cu MAC-uri fixe (= prod MACs). Asteapta ~60s pentru boot.

PASUL 2: Aplicare configuratii Talos
task dr:apply-talos-configs
# Scan nmap 10.57.57.0/24:50000, match MAC-config, talosctl apply --insecure
# Asteapta ~3-5 min pentru reboot la IP-uri statice (.80/.82/.84)

PASUL 3: Bootstrap cluster
task bootstrap:talos
# talhelper genconfig → apply → bootstrap etcd → kubeconfig local

PASUL 4: Instalare componente de baza
task bootstrap:apps
# helmfile: Cilium → CoreDNS → spegel → cert-manager → flux-operator → flux-instance

PASUL 5: Restaurare volume Longhorn din S3
task longhorn:restore
# BackupTarget → sync S3 → restore Volume CRDs → wait replicas → pvs.yaml → reconcile

PASUL 6: Flux preia controlul automat
task reconcile
# Flux sync → toate HelmRelease-urile se aplica. Asteapta ~5 min: kubectl get pods -A

PASUL 7: Post-restore Jellyfin
task longhorn:post-restore-jellyfin
# Extrage API key din DB, triggereaza refresh metadata. Imagini apar in ~10 min.

VERIFICARE
make dr-verify-phase2 # verifica K8s: pods, PVCs, certificate, endpoints
```

► Fisiere critice — TREBUIE backup-ate

Fisier

Importanta

age.key

<code>vps/.vault_pass</code>	CRITIC — cheia privata SOPS. Fara ea nu poti decripta nimic din repo
<code>talos/talsecret.sops.yaml</code>	CRITIC — parola Ansible Vault. Fara ea nu poti rula nici un playbook
<code>kubeconfig</code>	CRITIC — secretele Talos (CA, etcd key). Pierdere = rebuild complet
<code>talos/clusterconfig/</code>	Regenerabil (task bootstrap:talos), dar util
<code>github-deploy.key</code>	Regenerabil (talhelper genconfig), dar util
<code>github-deploy.key</code>	Flux SSH key pentru accesul la repo Git privat — regenerabil

Documentatie generata din repo-ul infrastructure la 07.06.2026. Pentru detalii suplimentare: README.md, DEPLOY.md, DR.md.